private data from peers the private data was disseminated to. If this value is set to 0, the private data is not disseminated at endorsement time, forcing private data pulls against endorsing peers on all authorised peers at commit time.

blockToLive: This represents how long the data should live on the private database in terms of blocks. The data will live for this specified number of blocks in the private database and after that it will get purged, making this data obsolete from the network. To keep private data indefinitely, that is, to never purge private data, set the *blockToLive* property to 0.

memberOnlyRead: A value of *true* indicates that peers automatically enforce that only clients belonging to one of the collection member organisations are allowed read access to private data. If a client from a non-member organisation attempts to execute a chaincode function that performs a read of a private data, the chaincode invocation is terminated with an error. Utilise the value of *false* if you would like to encode more granular access control within individual chaincode functions.

Here is a sample collection definition JSON file, containing an array of two collection definitions:

```
[
 {
"name": "collection1",
"policy": "OR('Org1MSP.member', 'Org2MSP.member')",
"requiredPeerCount": 0,
"maxPeerCount": 3,
"blockToLive":1000000,
"memberOnlyRead": true
 },
 {
"name": "collectionPrivate",
"policy": "OR('Org1MSP.member')",
"requiredPeerCount": 0,
"maxPeerCount": 3,
"blockToLive":3,
"memberOnlyRead": true
 }
]
```

Collection members may decide to share the private data with other parties if they get into a dispute or if they want to transfer the asset to a third party. The third party can then compute the hash of the private data and see if it matches the state on the channel ledger, proving that the state existed between the collection members at a certain point in time.

Using a collection within a channel versus a separate channel

- Use channels when entire transactions (and ledgers) must be kept confidential within a set of organisations that are members of the channel.
- Use collections when transactions (and ledgers) must be shared among a set of organisations, but when only a subset of those organisations should have access to some (or all) of the data within a transaction.

Additionally, since private data is disseminated peer-to-peer rather than via blocks, use private data collections when transaction data must be kept confidential from ordering service nodes.

Endorsement

Since private data is not included in the transactions that get submitted to the ordering service, and therefore is not included in the blocks that get distributed to all peers in a channel, the endorsing peer plays an important role in disseminating private data to other peers of authorised organisations. This ensures the availability of private data in the channel's collection, even if endorsing peers become unavailable after their endorsement. To assist with this dissemination, the *maxPeerCount* and *requiredPeerCount* properties in the collection definition control the degree of dissemination at endorsement time.

If the endorsing peer cannot successfully disseminate the private data to at least the *requiredPeerCount*, it will return an error back to the client. The endorsing peer will attempt to disseminate the private data to peers of different organisations, in an effort to ensure that each authorised organisation has a copy of the private data. Since transactions are not committed at chaincode execution time, the endorsing peer and recipient peers store a copy of the private data in a local transient store alongside their blockchain until the transaction is committed.

Committing private data

When authorised peers do not have a copy of the private data in their transient data store at commit time, either because they were not endorsing peers or because they did not receive the private data via dissemination at endorsement time, they will attempt to pull the private data from another authorised peer for a configurable amount of time, based on the peer property *peer.gossip.pvtData.pullRetryThreshold* in the peer configuration *core.yaml* file Therefore, it is important to set the *requiredPeerCount* and *maxPeerCount* properties large enough to ensure the availability of private data in your channel.

Referencing collections from chaincode

A set of shim APIs are available for setting and retrieving private data. The same chaincode data operations can be applied to channel state data and private data, but in the case of private data, a collection name is specified along with the data in the chaincode APIs — for example, *PutPrivateData(collection,key,value*) and *GetPrivateData(collection,key).*

Passing private data in a chaincode proposal

Since the chaincode proposal gets stored on the blockchain, it is also important not to include private data in the main part of the chaincode proposal. A special field in the chaincode

Continued On Page...97